

Projet du cours « Compilation »

Jalon 1 : Analyse lexicale et syntaxique de HOPIX

version 1.0

1 Spécification de la grammaire

1.1 Notations extra-lexicales

Les commentaires, espaces, les tabulations et les sauts de ligne jouent le rôle de séparateurs. Leur nombre entre les différents symboles terminaux peut donc être arbitraire. Ils sont ignorés par l'analyse lexicale : ce ne sont pas des lexèmes.

Les commentaires sont entourés des deux symboles « `{*}` » et « `*}` ». Par ailleurs, ils peuvent être imbriqués. Par ailleurs, le symbole « `##` » introduit un commentaire d'une ligne : tout ce qui suit ce symbole est ignoré jusqu'à la fin de la ligne.

1.2 Symboles

Symboles terminaux Les terminaux de Hopix sont répartis en trois catégories : les mots-clés, les identificateurs et la ponctuation.

Les mots-clés sont les mots réservés aux constructions du langage. Dans ce document, ils seront écrits avec une police grasse. Par exemple, **if** et **while** sont des mots-clé.

Les identificateurs sont les terminaux auxquels une information est associée. Ils se subdivisent en plusieurs classes : les identificateurs de variables, d'étiquettes, de constructeurs de données, de constructeurs de types, ainsi que les littéraux. Les littéraux comprennent les entiers, les caractères et les chaînes de caractères. Les identificateurs seront désignés en utilisant une police sans empattement (*sans serif* en anglais). Par exemple, les classes `type_con` ou `int` désignent respectivement les constructeurs de type et les entiers littéraux. Les identificateurs sont définis par les expressions rationnelles ci-dessous.

<code>var_id</code> $\equiv$ <code>[a-z] [A-Z a-z 0-9 _]*</code>	<i>Identificateur de variables préfixe</i>
<code>constr_id</code> $\equiv$ <code>[A-Z] [A-Z a-z 0-9 _]*</code>	<i>Identificateur de constructeurs de données</i>
<code>label_id</code> $\equiv$ <code>[a-z] [A-Z a-z 0-9 _]*</code>	<i>Identificateur d'étiquettes d'enregistrement</i>
<code>type_con</code> $\equiv$ <code>[a-z] [A-Z a-z 0-9 _]*</code>	<i>Identificateur de constructeurs de type</i>
<code>type_variable</code> $\equiv$ <code>`[a-z] [A-Z a-z 0-9 _]*</code>	<i>Identificateur de variables de type</i>
<code>int</code> $\equiv$ <code>-?[0-9] 0x[0-9 a-f A-F]⁺ 0b[0-1]⁺ 0o[0-7]⁺</code>	<i>Littéraux entiers</i>
<code>char</code> $\equiv$ <code>'atom'</code>	<i>Littéraux caractères</i>
<code>string</code> $\equiv$ <code>"((atom ' \") -{ " })* "</code>	<i>Littéraux chaîne de caractères</i>
<code>atom</code> $\equiv$ <code>\000 ... \255 \0x[0-9 a-f A-F]² [printable] -{ ' } \ \n \t \b \r</code>	

Autrement dit, les identificateurs de valeurs, de constructeurs de type et de champs d'enregistrement commencent par une lettre minuscule et peuvent comporter ensuite des majuscules, des minuscules, des chiffres et le caractère souligné `_`. Les identificateurs de constructeurs de données peuvent comporter les mêmes caractères, mais doivent commencer par une majuscule. Les variables de type doivent débuter par un guillemet oblique.

Les entiers littéraux sont constituées de chiffres en notation décimale, en notation hexadécimale, en notation binaire ou en notation octale. Les entiers utilisent une représentation binaire sur 64 bits en complément à deux. Les constantes entières sont donc prises dans $[-2^{63}; 2^{63} - 1]$.

Les caractères littéraux sont écrits entre guillemets simples (ce qui signifie en particulier que les guillemets simples doivent être échappés dans les constantes de caractères). On y trouve tous les symboles ASCII affichables (voir la spécification de ASCII pour plus de détails). Par ailleurs, sont des caractères valides : les séquences d'échappement usuelles ; les séquences d'échappement de trois chiffres décrivant le code ASCII du caractère en notation décimale ; les séquences d'échappement de deux chiffres décrivant le code ASCII en notation hexadécimale.

Les chaînes de caractères littérales sont formées d'une séquence de caractères. Cette séquence est entourée de guillemets (ce qui signifie en particulier que les guillemets doivent être échappés dans les chaînes). On peut y faire figurer un caractère par son code ASCII décrit via le même mécanisme que pour les caractères littéraux.

La ponctuation, comme les identificateurs, sera notée avec une police à chasse fixe. Par exemple « `(` » et « `=` » appartiennent à la ponctuation.

Symboles non-terminaux Les symboles non-terminaux seront notés à l'aide d'une police italique, comme par exemple *expr*.

Une séquence entre crochets est optionnelle, comme par exemple « `[ ref ]` ». Attention à ne pas confondre ces crochets avec les symboles terminaux de ponctuation notés `[` et `]`. Une séquence entre accolades se répète zéro fois ou plus, comme par exemple « `( arg { , arg } )` ».

2 Grammaire en format BNF

La grammaire du langage est spécifiée à l'aide du format BNF.

Programme Un programme est constitué d'une séquence de définitions de types et de valeurs.

$p ::= \{ \text{definition} \}$	<i>Programme</i>
$\text{definition} ::= \text{type } \text{type_con} [< \text{type_variable} \{ , \text{type_variable} \} >] [= \text{tdefinition}]$ $\quad \text{extern } \text{var_id} : \text{type_scheme}$ $\quad \text{vdefinition}$	<i>Définition de type</i> <i>Valeurs externes</i> <i>Définition de valeur(s)</i>
$\text{tdefinition} ::= [\mid] \text{constr_id} [(\text{type} \{ , \text{type} \})]$ $\quad \{ \mid \text{constr_id} [(\text{type} \{ , \text{type} \})] \}$ $\quad \{ \text{label_id} : \text{type} \{ , \text{label_id} : \text{type} \} \}$	<i>Type somme</i> <i>Type produit étiqueté</i>
$\text{vdefinition} ::= \text{let } \text{var_id} [: \text{type_scheme}] = \text{expr}$ $\quad \text{fun } \text{fundef} \{ \text{and } \text{fundef} \}$	<i>Valeur simple</i> <i>Fonction(s)</i>
$\text{fundef} ::= [: \text{type_scheme}] \text{var_id } \text{pattern} = \text{expr}$	

Types de données La syntaxe des types est donnée par la grammaire suivante :

$\text{type} ::= \text{type_con} [< \text{type} \{ , \text{type} \} >]$	<i>Application d'un constructeur de type</i>
$\quad \text{type} \rightarrow \text{type}$	<i>Fonctions</i>
$\quad \text{type} * \text{type} \{ * \text{type} \}$	<i>N-uplets (N > 1)</i>
$\quad \text{type_variable}$	<i>Variables de type</i>
$\quad (\text{type})$	<i>Type entre parenthèses</i>
$\text{type_scheme} ::= [[\text{type_variable} \{ , \text{type_variable} \}]] \text{type}$	

Expression La syntaxe des expressions du langage est donnée par la grammaire suivante.

$\text{expr} ::= \text{int}$	<i>Entier positif</i>
$\quad \text{char}$	<i>Caractère</i>
$\quad \text{string}$	<i>Chaîne de caractères</i>
$\quad \text{var_id} [< [\text{type} \{ , \text{type} \}] >]$	<i>Variable</i>
$\quad \text{constr_id} [< [\text{type} \{ , \text{type} \}] >] [(\text{expr} \{ , \text{expr} \})]$	<i>Construction d'une donnée</i>
$\quad ()$	<i>Construction d'un 0-uplet</i>
$\quad (\text{expr} , \text{expr} \{ , \text{expr} \})$	<i>Construction d'un n-uplet (n > 1)</i>
$\quad \{ \text{label_id} = \text{expr} \{ , \text{label_id} = \text{expr} \} \} [< [\text{type} \{ , \text{type} \}] >]$	<i>Construction d'un enregistrement</i>
$\quad \text{expr} . \text{label_id} [< [\text{type} \{ , \text{type} \}] >]$	<i>Projection d'un champ</i>
$\quad \text{expr} ; \text{expr}$	<i>Séquencement</i>
$\quad \text{vdefinition} ; \text{expr}$	<i>Définition locale</i>
$\quad \backslash \text{pattern} \rightarrow \text{expr}$	<i>Fonction anonyme</i>
$\quad \text{expr } \text{expr}$	<i>Application</i>
$\quad \text{expr } \text{binop } \text{expr}$	<i>Application infixe</i>
$\quad \text{match } (\text{expr}) \{ \text{branches} \}$	<i>Analyse de motifs</i>
$\quad \text{if } (\text{expr}) \text{ then } \{ \text{expr} \} [\text{else } \{ \text{expr} \}]$	<i>Conditionnelle</i>
$\quad \text{ref } \text{expr}$	<i>Allocation</i>
$\quad \text{expr} := \text{expr}$	<i>Affectation</i>
$\quad ! \text{expr}$	<i>Lecture</i>
$\quad \text{while } (\text{expr}) \{ \text{expr} \}$	<i>Boucle non bornée</i>
$\quad \text{do } \{ \text{expr} \} \text{ until } (\text{expr})$	<i>Boucle non bornée et non vide</i>
$\quad \text{for } \text{var_id} \text{ from } (\text{expr}) \text{ to } (\text{expr}) \{ \text{expr} \}$	<i>Boucle bornée</i>
$\quad (\text{expr})$	<i>Parenthésage</i>
$\quad (\text{expr} : \text{type})$	<i>Annotation de type</i>

Voici la grammaire des définitions auxiliaires utilisées par la grammaire des expressions :

$binop ::= + \mid - \mid * \mid / \mid \&\& \mid \mid =? \mid <=? \mid >=? \mid <? \mid >?$	<i>Opérateurs binaires</i>
$branches ::= [\mid] \text{ branch } \{ \mid \text{ branch } \}$	<i>Liste de cas</i>
$branch ::= pattern \rightarrow expr$	<i>Cas d'analyse</i>

Motifs Les motifs (*patterns* en anglais), utilisés par l'analyse de motifs, ont la syntaxe suivante :

$pattern ::= var_id$	<i>Motif universel liant</i>
$ _$	<i>Motif universel non liant</i>
$ ([pattern \{ , pattern \}])$	<i>N-uplets ou parenthésage</i>
$ pattern : type$	<i>Annotation de type</i>
$ int$	<i>Entier</i>
$ char$	<i>Caractère</i>
$ string$	<i>Chaîne de caractères</i>
$ constr_id [< [type \{ , type \}] >] [(pattern \{ , pattern \})]$	<i>Valeurs étiquetées</i>
$ \{ label_id = pattern \{ , label_id = pattern \} \} [< [type \{ , type \}] >]$	<i>Enregistrement</i>
$ pattern \mid pattern$	<i>Disjonction</i>
$ pattern \& pattern$	<i>Conjonction</i>

Remarques Notez bien que cette grammaire de Hopix est ambiguë ! Vous devez fixer des priorités entre les différentes constructions ainsi que des associativités aux différents opérateurs. *In fine*, c'est la batterie de tests en ligne qui vous permettra de valider vos choix. Cependant, il est fortement conseillé de poser des questions sur la liste de diffusion du cours pour obtenir des informations supplémentaires sur les règles de disambiguation associées à cette grammaire.

3 Code fourni

Un squelette de code vous est fourni, il est disponible sur le dépôt Git du cours.

<https://moule.informatique.univ-paris-diderot.fr/guatto/compilation-m1-2024>

Vous devez vous connecter à ce serveur Gitlab et vous créer un dépôt par branchement (*fork*) du projet `compilation-m1-2024`. **Il doit être privé**. L'arbre de sources contient les modules OCaml à compléter.

La commande `dune build` produit un exécutable appelé `flap.exe` et situé dans le répertoire `src`. On peut aussi l'exécuter avec la commande `dune exec ./src/flap.exe -- OPTIONS` depuis la racine de Flap. On doit pouvoir l'appeler avec un nom de fichier en argument. En cas de réussite (de l'analyse syntaxique), le code de retour de ce programme doit être 0. Dans le cas d'un échec, le code de retour doit être 1.

4 Travail à effectuer

La première partie du projet est l'écriture de l'analyseur lexical et de l'analyseur syntaxique spécifiés par la grammaire précédente. Le projet est à rendre **avant le** :

20 octobre 2025 à 19h59

Le rendu est automatique si vous avez suivi la procédure décrite ci-dessus.

Pour finir, vous devez vous assurer des points suivants :

- Le projet contenu dans ce dépôt **doit compiler**.
- Vous devez **être les auteurs** de ce projet.
- Il doit être rendu **à temps**.

Si l'un de ces points n'est pas respecté, la note de 0 vous sera affectée.

5 Log

29-09-2025 Version initiale.